

RECRUITAI

PROJECT SPECIFICATION & IMPLEMENTATION BLUEPRINT

Taskify API

Detailed project brief generated from the project record.

DURATION

14 days

TECHNOLOGY STACK

Node.js

Express.js

MongoDB

Project Overview

Summary of the problem, purpose and expected outcome.

PROJECT SUMMARY

Build a RESTful API for a simple task management service where users can create, view, update and delete personal tasks.

TYPE

backend

DURATION

14 days

Primary Objectives

- Implement a clean Express.js server
- Design a MongoDB schema for tasks
- Expose REST endpoints for task CRUD
- Add input validation and error handling
- Write unit and integration tests
- Provide a Dockerfile for local deployment

Technology Stack

Technologies, frameworks and tools expected for implementation.

Node.js

Technology / Tool

Express.js

Technology / Tool

MongoDB

Technology / Tool

Functional Requirements

- Create a task with title, description, dueDate and status.
- Retrieve a paginated list of a user's tasks.
- Update any field of an existing task.
- Delete a task permanently.

- Filter tasks by status (pending, completed).
- Return proper HTTP status codes for success and error cases.

Non-Functional Requirements

- Validate all request bodies and return 400 on invalid data.
- Handle unexpected errors with a generic 500 response.
- All responses must be JSON and complete within 200 ms for typical loads.
- Store tasks in MongoDB with appropriate indexes on userId and status.
- Document all endpoints using OpenAPI 3.0.

Modules & Features

Major functional areas that make up the project.

API Layer

Express routes and middleware.

- Route definitions
- Request parsing
- Response formatting
- Error middleware

Service Layer

Business logic for task operations.

- CreateTask
- GetTasks
- UpdateTask
- DeleteTask

Data Access Layer

Mongoose models and DB queries.

- Task schema
- CRUD queries
- Index creation

Utilities

Common helpers such as validation and logging.

- Joi validation schemas
- Logger wrapper
- Env config

Implementation Roadmap

Recommended phases and milestones for completing the project.

1

Foundation

Initialize npm project, Add Express, Mongoose, Joi, Configure ESLint and Prettier, Create Dockerfile, Set up Jest, Create basic folder layout

- Initialize npm project
- Add Express, Mongoose, Joi
- Configure ESLint and Prettier
- Create Dockerfile
- Set up Jest

2

Data Model

Design Task schema, Add indexes on userId and status, Write model unit tests, Create seed script, Document schema in markdown, Integrate model with Express app

- Design Task schema
- Add indexes on userId and status
- Write model unit tests
- Create seed script
- Document schema in markdown

3

Core CRUD

Create route files, Implement Service functions, Wire routes to services, Return proper status codes, Add basic request logging, Test endpoints with Postman collection

- Create route files
- Implement Service functions
- Wire routes to services
- Return proper status codes
- Add basic request logging

4

Validation & Security

Define Joi schemas, Add validation middleware, Implement simple API key auth middleware, Return 401 for missing/invalid key, Update error handler, Write validation tests

- Define Joi schemas
- Add validation middleware
- Implement simple API key auth middleware
- Return 401 for missing/invalid key
- Update error handler

5

Testing & Deployment

Write unit tests for services, Write integration tests for routes, Add coverage script, Generate OpenAPI spec, Build Docker image, Run smoke test in container

- Write unit tests for services
- Write integration tests for routes
- Add coverage script
- Generate OpenAPI spec
- Build Docker image

API Design

Key API endpoints and expected responsibilities.

POST

/api/tasks

Create a new task

GET

/api/tasks

List tasks with pagination and optional status filter

GET

/api/tasks/:id

Get a single task

PUT

/api/tasks/:id

Update task fields

DELETE

/api/tasks/:id

Delete a task

Evaluation Criteria

Criteria that can be used to assess the completed project.

- Criterion: Correctness | Description: All CRUD endpoints work as specified | Weight: 25
- Criterion: Code Quality | Description: Readable, modular code with linting passed | Weight: 15
- Criterion: Validation & Error Handling | Description: Invalid input returns 400, server errors 500 | Weight: 15
- Criterion: Testing Coverage | Description: "e80% unit & integration test coverage | Weight: 15
- Criterion: Documentation | Description: OpenAPI spec and README complete | Weight: 15
- Criterion: Deployment | Description: Dockerfile builds and runs without errors | Weight: 15

Expected Deliverables

Artifacts and outcomes expected at project completion.

- Source code repository
- README with setup instructions
- OpenAPI 3.0 spec file
- Dockerfile and docker compose.yml
- Test suite with coverage report
- Database schema document

Candidate Alignment

Project requirements compared with the candidate information available in the project record.

The candidate knows Node.js, Express.js and MongoDB, which are exactly the stack required. A 14 day junior project focuses on core API design, basic validation, testing and Docker deployment—areas a junior backend developer must master without overwhelming complexity.

Implementation Note

This document is generated from the project record. AI-generated requirements should be reviewed by an administrator before implementation.